

Knowledge Representation Project — Family/Kinship Ontology in Description Logic

A single project that ties together the whole course arc: Logic review (Unit 1) → Knowledge & DL foundations (Unit 2, AL/ALN/ALC) → Tableau (Unit 3) → OWL/Protégé (the lab units). Domain kept small on purpose (family/kinship relations) so every construct taught in the slides gets used at least once, without the project ballooning.

0. How this maps to your syllabus

Project section	Course unit
§1 Domain & TBox in AL/ALN	U2 L2.1-L2.3 (Knowledge, structural DL, AL/ALN)
§2 Extending to ALC	U2 L2.3, L2.6 (Beyond ALN, ALC semantics)
§3 Extending to ALCQ	U3 L3.2/L3.3 (numerals, qualified number restrictions)
§4 ABox	U2 L2.5 (T-Box and A-Box)
§5 Reasoning tasks	U2 L2.2 (SAT, subsumption, equivalence, disjointness)
§6 Tableau derivations	U3 L3.1/L3.2 (Tableau method, rules, extensions)
§7 OWL/Protégé	Lab: "Classwork on Protégé"

1. Domain, scope and the AL/ALN TBox

Domain: family/kinship relations — the textbook DL domain, chosen because it lets subsumption, satisfiability, and disjointness all have intuitive, checkable answers.

Signature

- Primitive concepts: `Person`, `Male`
- Primitive role: `hasChild`

TBox — Stage 1 (AL / ALN only: atomic negation, unqualified $\exists R$, $\forall R.C$, number restrictions, no union, no qualified existential)

Female	$\doteq$	$\neg \text{Male}$	
Parent	$\doteq$	$\text{Person} \sqcap \exists \text{hasChild}$	(unqualified existential – AL)
Childless	$\doteq$	$\text{Person} \sqcap \forall \text{hasChild}.\perp$	(value restriction to bottom)
BigFamily	$\doteq$	$\text{Parent} \sqcap (\geq 3 \text{ hasChild})$	(ALN number restriction)
SmallFamily	$\doteq$	$\text{Parent} \sqcap (\leq 2 \text{ hasChild})$	

At this stage we already cannot express "a parent whose children are all doctors" (needs qualified $\forall$ on a *defined* concept, fine) or "a woman who has at least one child that is a person" with a *qualified* existential — AL's $\exists\text{hasChild}$ is unqualified ($\equiv \exists\text{hasChild}.\top$). That limitation is exactly why the course moves to ALC next.

2. Extending the TBox to ALC (full negation, union, qualified existential)

Woman	$\equiv$	Female $\sqcap$ Person	
Man	$\equiv$	Male $\sqcap$ Person	
Mother	$\equiv$	Woman $\sqcap$ $\exists\text{hasChild}.\text{Person}$	(qualified $\exists$ — needs E)
Father	$\equiv$	Man $\sqcap$ $\exists\text{hasChild}.\text{Person}$	
Grandparent	$\equiv$	Parent $\sqcap$ $\exists\text{hasChild}.\text{Parent}$	
HappyParent	$\equiv$	Parent $\sqcap$ $\forall\text{hasChild}.\text{(Doctor } \sqcup \text{ Lawyer)}$	(needs union — U)
Bachelor	$\equiv$	Man $\sqcap$ $\neg\exists\text{hasSpouse}.\text{Person}$	(needs full negation of a complex concept — C)

Doctor and Lawyer are additional primitive concepts (both $\sqsubseteq \text{Person}$), and hasSpouse an additional primitive role.

3. Extending to ALCQ (qualified number restrictions)

ParentOfTwoSons	$\equiv$	Parent $\sqcap$ (≥ 2 hasChild.Male) $\sqcap$ (≤ 2 hasChild.Male)
SingleParent	$\equiv$	Parent $\sqcap$ (≤ 0 hasSpouse)

$(\geq 2 \text{ hasChild.Male})$ is a *qualified* number restriction (counts only children satisfying Male) — this is what separates ALCQ from the unqualified ALN numerals in §1.

4. ABox

Male(john)	Male(tom)	
Female(mary)	Female(ann)	
Person(john)	Person(mary)	Person(tom) Person(ann)
Doctor(tom)		
hasChild(john, tom)	hasChild(john, ann)	
hasChild(mary, tom)	hasChild(mary, ann)	

5. Reasoning tasks (competency questions, DL-flavoured)

1. **Satisfiability:** is Grandparent satisfiable?
2. **Subsumption:** does $\text{Mother} \sqsubseteq \text{Parent}$ hold?
3. **Disjointness:** are Bachelor and Father disjoint?
4. **Equivalence:** are Parent and $\text{Mother} \sqcup \text{Father}$ equivalent? (they should NOT be — a parent need not be identified by gender)

5. **Instance checking:** given the ABox, is `mary` an instance of `Mother`? Is `john` an instance of `Grandparent`?

6. **Realization:** what is the most specific concept each ABox individual belongs to?

6. Tableau derivations (the theory centrepiece)

6.1 Proving `Mother $\sqsubseteq$ Parent` (closed tableau $\Rightarrow$ subsumption holds)

Subsumption `Mother  $\sqsubseteq$  Parent` holds iff `Mother  $\sqcap$   $\neg$ Parent` is **unsatisfiable**. Unfold definitions and negate:

$\neg \text{Parent} = \neg (\text{Person} \sqcap \exists \text{hasChild}) = \neg \text{Person} \sqcup \forall \text{hasChild}.\perp$ (De Morgan)

Initial node: `(Female  $\sqcap$  Person  $\sqcap$   $\exists$ hasChild.Person)  $\sqcap$  ( $\neg$ Person  $\sqcup$   $\forall$ hasChild. $\perp$ ) (x)`

Step 1 – $\sqcap$ -rule (split the outer intersection):

Female(x)
Person(x)
 $\exists$ hasChild.Person(x)
 $(\neg \text{Person} \sqcup \forall \text{hasChild}.\perp)(x)$

Step 2 – $\sqcup$ -rule (nondeterministic branch on the disjunction):

Branch A: $\neg \text{Person}(x)$

Clash: Person(x) and $\neg \text{Person}(x)$ both hold $\rightarrow$ BRANCH CLOSES $\times$

Branch B: $\forall \text{hasChild}.\perp(x)$

Step 3 – $\exists$ -rule on $\exists$ hasChild.Person(x):

introduce fresh individual y
hasChild(x, y)
Person(y)

Step 4 – $\forall$ -rule: since hasChild(x,y) and $\forall \text{hasChild}.\perp(x)$ both hold:

$\perp(y)$

Clash: $\perp(y)$ is never satisfiable $\rightarrow$ BRANCH CLOSES $\times$

Both branches closed $\Rightarrow$ `Mother  $\sqcap$   $\neg$ Parent` is unsatisfiable $\Rightarrow$ `Mother  $\sqsubseteq$  Parent` holds.

6.2 Checking `Grandparent` is satisfiable (open tableau $\Rightarrow$ SAT)

Initial node: `Grandparent(x)` $\rightarrow$ unfold $\rightarrow$ `(Person  $\sqcap$   $\exists$ hasChild)  $\sqcap$   $\exists$ hasChild.Parent (x)`

Step 1 – Π -rule:

Person(x)
 $\exists \text{hasChild}(x)$ (unqualified, from Parent's definition)
 $\exists \text{hasChild}.\text{Parent}(x)$

Step 2 – $\exists$ -rule on $\exists \text{hasChild}.\text{Parent}(x)$:

introduce fresh individual y
hasChild(x, y)
Parent(y)

Step 3 – unfold Parent(y):

Person(y)
 $\exists \text{hasChild}(y) \rightarrow$ introduce fresh z, hasChild(y, z)

No rule is applicable and no clash exists anywhere in the model {x, y, z}.

Tableau is complete with no clash $\Rightarrow$ **Grandparent** is satisfiable (a canonical model $x \rightarrow y \rightarrow z$, a 3-generation chain, witnesses it).

6.3 Reading the two proofs together

- §6.1 is the pattern the slides call "compatibility of an A-box with an expression" (L3.1): closing every branch proves *non*-existence of a model, i.e. subsumption.
- §6.2 shows the complementary case: one open, clash-free branch is enough to prove satisfiability — you don't need every branch to stay open, just one.

7. From TBox/ABox to OWL — build it in Protégé

Manchester syntax (paste directly into Protégé's class expression editor):

Class: Person
Class: Male
Class: Female
 EquivalentTo: not Male

Class: Parent
 EquivalentTo: Person and (hasChild some Thing)

Class: Woman
 EquivalentTo: Female and Person

Class: Man
 EquivalentTo: Male and Person

Class: Mother
 EquivalentTo: Woman and (hasChild some Person)

Class: Father
 EquivalentTo: Man and (hasChild some Person)

Class: Grandparent
 EquivalentTo: Parent and (hasChild some Parent)

Class: HappyParent
 EquivalentTo: Parent and (hasChild only (Doctor or Lawyer))

Class: Bachelor
 EquivalentTo: Man and not (hasSpouse some Person)

Class: ParentOfTwoSons
 EquivalentTo: Parent and (hasChild min 2 Male) and (hasChild max 2 Male)

Class: SingleParent
 EquivalentTo: Parent and (hasSpouse max 0 Thing)

ObjectProperty: hasChild
ObjectProperty: hasSpouse
 Characteristics: Symmetric

Individual: john
 Types: Male, Person
 Facts: hasChild tom, hasChild ann

Individual: mary
 Types: Female, Person
 Facts: hasChild tom, hasChild ann

Individual: tom
 Types: Male, Person, Doctor

Individual: ann
 Types: Female, Person

Build & verify plan:

1. Enter the classes/properties/individuals above in Protégé (Entities tabs, or the OWL Manchester Syntax editor directly).
2. Run the **HermiT** reasoner (Reasoner menu → Start reasoner → Synchronize).
3. Check the inferred class hierarchy: confirm Mother is shown as a subclass of Parent — this is the automated confirmation of the hand tableau proof in §6.1.
4. Check individuals: confirm mary is inferred as an instance of Mother and Grandparent (realization, from the ABox facts $\text{hasChild}(\text{mary}, \text{tom})$, $\text{hasChild}(\text{mary}, \text{ann})$, plus tom / ann both being Person), then extend the ABox with one more generation to actually witness Grandparent if you want that inference to fire).
5. Use the **DL Query** tab to directly ask the reasoning-task questions from §5 (e.g., query $\text{Mother and not Parent}$ → should return "unsatisfiable", mirroring §6.1).

8. What to hand in / present

1. **Theory writeup** (§1–§6): the TBox stages, ABox, and the two full tableau derivations — this is the part that shows you can do the algorithm by hand, not just trust the reasoner.
2. **Protégé file** (.owl): the built ontology from §7.
3. **Screenshots**: inferred class hierarchy (showing $\text{Mother} \sqsubseteq \text{Parent}$) inferred, matching §6.1) and one DL Query result.
4. **Short discussion** (half a page): compare the hand tableau proof to what HermiT reports — do they agree? Note that HermiT implements a tableau-based decision procedure itself, so this is literally the same algorithm you ran by hand.

9. Optional extension (if you want to reach into Unit 4)

Unit 4 moves past ALC into ALCQIreg (regular role expressions, inverses, role reification) and shows how satisfiability there is decided by translating into **2-way alternating tree automata (2ATA)** rather than tableau, because tableau alone doesn't terminate once you add regular-expression role composition. As a bonus section you could take one construct from §3 (e.g. SingleParent 's $(\leq 0 \text{ hasSpouse})$) and briefly discuss how it would be represented in ALCFIREG via **role reification** ($(\vee 1 \circ \text{id}(A_R) \circ \vee 2)$, L4.2) instead of a native number restriction — this is a discussion point, not something you need to implement.